Practical 9

Part A:

Aim:

1. Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.

Code:

CounterRace.java

```
public class CounterRace {

    static final int THREADS = 8;
    static final int INCREMENTS = 500_000;

    static class Counter {
        int value = 0;

        // Unsafe: multiple threads can overwrite each other's
updates
        void incrementUnsafe() {
            value++;
        }

        // Safe: only one thread can execute this method at a
time
        synchronized void incrementSafe() {
            value++;
        }
    }
```

```
    public static void main(String[] args) throws
InterruptedException {

        Counter unsafeCounter = new Counter();
        Thread[] unsafeThreads = new Thread[THREADS];

        long start = System.nanoTime();

        for (int i = 0; i < THREADS; i++) {
            unsafeThreads[i] = new Thread(() -> {
                for (int j = 0; j < INCREMENTS; j++) {
                    unsafeCounter.incrementUnsafe();
                }
            });
            unsafeThreads[i].start();
        }

        for (Thread thread : unsafeThreads) {
            thread.join();
        }

        long unsafeTime = System.nanoTime() - start;

        Counter safeCounter = new Counter();
        Thread[] safeThreads = new Thread[THREADS];

        start = System.nanoTime();

        for (int i = 0; i < THREADS; i++) {
            safeThreads[i] = new Thread(() -> {
                for (int j = 0; j < INCREMENTS; j++) {
                    safeCounter.incrementSafe();
                }
            });
            safeThreads[i].start();
```

```
        }

        for (Thread thread : safeThreads) {
            thread.join();
        }

        long safeTime = System.nanoTime() - start;

        int expected = THREADS * INCREMENTS;

        System.out.println("Expected count: " + expected);
        System.out.println("Without synchronization: "
                + unsafeCounter.value);
        System.out.println("With synchronization: "
                + safeCounter.value);

        System.out.printf("Unsafe time: %.2f ms%n",
                unsafeTime / 1_000_000.0);
        System.out.printf("Safe time: %.2f ms%n",
                safeTime / 1_000_000.0);
    }
}
```

Output:



2. Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do.

Code:

SeatBookingRace.java

```
import java.util.concurrent.CountDownLatch;

public class SeatBookingRace {

    static final int INITIAL_SEATS = 5;
    static final int CUSTOMERS = 10;

    static class BookingSystem {
        private int seatsLeft = INITIAL_SEATS;
        private int successfulBookings = 0;

        // Demonstration only: deliberately unsafe
        boolean bookUnsafe(CountDownLatch checked)
                throws InterruptedException {

            // Each thread reads the shared state without a
lock
            int observedSeats = seatsLeft;

            // Ensure every customer has checked before updates
begin
            checked.countDown();
            checked.await();

            if (observedSeats > 0) {
                seatsLeft--;
                successfulBookings++;
                return true;
```

```
            }

            return false;
        }

        // Thread-safe booking
        synchronized boolean book() {
            if (seatsLeft > 0) {
                seatsLeft--;
                successfulBookings++;
                return true;
            }

            return false;
        }

        int getSeatsLeft() {
            return seatsLeft;
        }

        int getSuccessfulBookings() {
            return successfulBookings;
        }
    }

    public static void main(String[] args)
            throws InterruptedException {

        // Unsafe demonstration
        BookingSystem unsafe = new BookingSystem();
        CountDownLatch checked = new CountDownLatch(CUSTOMERS);
        Thread[] threads = new Thread[CUSTOMERS];

        for (int i = 0; i < CUSTOMERS; i++) {
            final int customer = i + 1;
```

```
            threads[i] = new Thread(() -> {
                try {
                    boolean booked =
unsafe.bookUnsafe(checked);
                    System.out.println("Unsafe customer " +
customer
                            + ": " + (booked ? "BOOKED" :
"REJECTED"));
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });

            threads[i].start();
        }

        for (Thread thread : threads) {
            thread.join();
        }

        System.out.println("\nUnsafe successful bookings: "
                + unsafe.getSuccessfulBookings());
        System.out.println("Unsafe seats left: "
                + unsafe.getSeatsLeft());

        // Safe demonstration
        BookingSystem safe = new BookingSystem();
        Thread[] safeThreads = new Thread[CUSTOMERS];

        for (int i = 0; i < CUSTOMERS; i++) {
            final int customer = i + 1;

            safeThreads[i] = new Thread(() -> {
                boolean booked = safe.book();
```

```
                System.out.println("Safe customer " + customer
                        + ": " + (booked ? "BOOKED" :
"REJECTED"));
            });

            safeThreads[i].start();
        }

        for (Thread thread : safeThreads) {
            thread.join();
        }

        System.out.println("\nSafe successful bookings: "
                + safe.getSuccessfulBookings());
        System.out.println("Safe seats left: "
                + safe.getSeatsLeft());
    }
}
```

Output:

```
Problems 1  Output  Debug Console  Terminal  Ports                Code - Part A

PS G:\OOP> cd "g:\OOP\Practical 9\Part A\" ; if ($?) { javac SeatBookingRace.java } ; if ($?) { java SeatBookingRace }
  Unsafe customer 3: BOOKED
  Unsafe customer 10: BOOKED
  Unsafe customer 2: BOOKED
  Unsafe customer 1: BOOKED
  Unsafe customer 4: BOOKED
  Unsafe customer 7: BOOKED
  Unsafe customer 8: BOOKED
  Unsafe customer 5: BOOKED
  Unsafe customer 9: BOOKED
  Unsafe customer 6: BOOKED

  Unsafe successful bookings: 10
  Unsafe seats left: -5
  Safe customer 1: BOOKED
  Safe customer 2: BOOKED
  Safe customer 3: BOOKED
  Safe customer 5: BOOKED
  Safe customer 6: BOOKED
  Safe customer 7: REJECTED
  Safe customer 4: REJECTED
  Safe customer 9: REJECTED
  Safe customer 10: REJECTED
  Safe customer 8: REJECTED

  Safe successful bookings: 5
  Safe seats left: 0
PS G:\OOP\Practical 9\Part A>
```

3. (For Advanced Learners – home) Split a large array of numbers across several threads that add into one shared total, show the wrong result, then fix it two ways and compare; decide how to measure the difference.

Code:

ParallelArraySum.java

```
public class ParallelArraySum {

    static final int SIZE = 2_000_000;
    static final int THREADS = 4;

    static class SharedTotal {
        long total = 0;

        void addUnsafe(long value) {
            total += value;
        }

        synchronized void addSafe(long value) {
            total += value;
        }
    }

    static long sumArray(int[] array) {
        long sum = 0;

        for (int value : array) {
            sum += value;
        }

        return sum;
    }
```

```
    static long runUnsafe(int[] array) throws
InterruptedException {
        SharedTotal total = new SharedTotal();
        Thread[] workers = new Thread[THREADS];

        for (int t = 0; t < THREADS; t++) {
            final int start = t * array.length / THREADS;
            final int end = (t + 1) * array.length / THREADS;

            workers[t] = new Thread(() -> {
                for (int i = start; i < end; i++) {
                    total.addUnsafe(array[i]);
                }
            });

            workers[t].start();
        }

        for (Thread worker : workers) {
            worker.join();
        }

        return total.total;
    }

    static long runSynchronized(int[] array)
            throws InterruptedException {

        SharedTotal total = new SharedTotal();
        Thread[] workers = new Thread[THREADS];

        for (int t = 0; t < THREADS; t++) {
            final int start = t * array.length / THREADS;
            final int end = (t + 1) * array.length / THREADS;
```

```
            workers[t] = new Thread(() -> {
                for (int i = start; i < end; i++) {
                    total.addSafe(array[i]);
                }
            });

            workers[t].start();
        }

        for (Thread worker : workers) {
            worker.join();
        }

        return total.total;
    }

    static long runLocalSums(int[] array)
            throws InterruptedException {

        long[] partialSums = new long[THREADS];
        Thread[] workers = new Thread[THREADS];

        for (int t = 0; t < THREADS; t++) {
            final int workerIndex = t;
            final int start = t * array.length / THREADS;
            final int end = (t + 1) * array.length / THREADS;

            workers[t] = new Thread(() -> {
                long localSum = 0;

                for (int i = start; i < end; i++) {
                    localSum += array[i];
                }

                // Each thread writes to its own array element
```

```
                partialSums[workerIndex] = localSum;
            });

            workers[t].start();
        }

        for (Thread worker : workers) {
            worker.join();
        }

        long total = 0;

        for (long partialSum : partialSums) {
            total += partialSum;
        }

        return total;
    }

    static long measure(String name, SumTask task)
            throws InterruptedException {

        long start = System.nanoTime();
        long result = task.run();
        long elapsed = System.nanoTime() - start;

        System.out.printf(
            "%-24s sum = %d, time = %.2f ms%n",
            name, result, elapsed / 1_000_000.0
        );

        return result;
    }

    @FunctionalInterface
```

```
    interface SumTask {
        long run() throws InterruptedException;
    }

    public static void main(String[] args) throws
InterruptedException {

        int[] array = new int[SIZE];

        // Fill with deterministic values
        for (int i = 0; i < SIZE; i++) {
            array[i] = 1;
        }

        long expected = sumArray(array);

        System.out.println("Array size: " + SIZE);
        System.out.println("Threads: " + THREADS);
        System.out.println("Expected sum: " + expected);
        System.out.println();

        long unsafe = measure("Unsafe shared total",
                () -> runUnsafe(array));

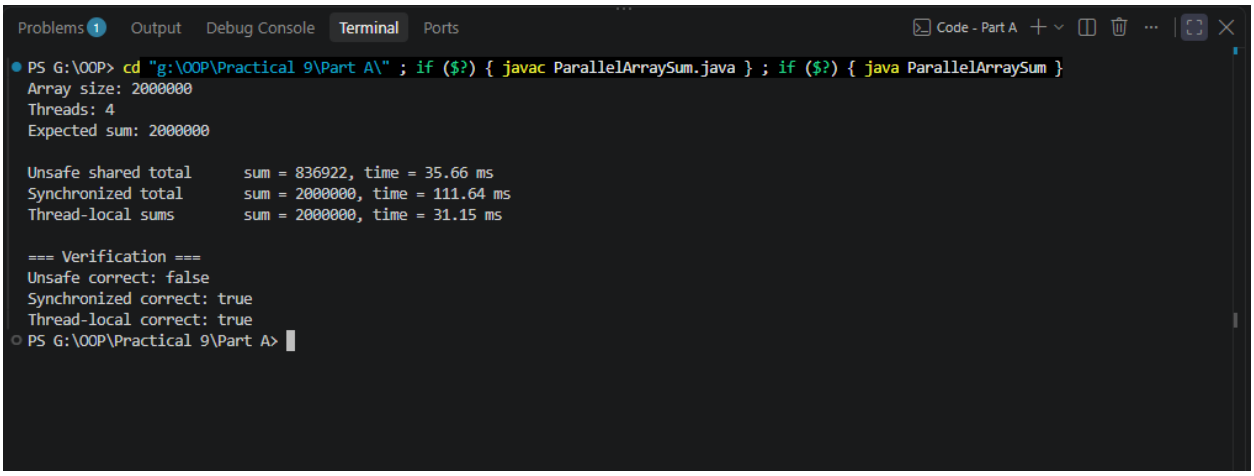
        long synchronizedSum = measure("Synchronized total",
                () -> runSynchronized(array));

        long local = measure("Thread-local sums",
                () -> runLocalSums(array));

        System.out.println("\n=== Verification ===");
        System.out.println("Unsafe correct: " + (unsafe ==
expected));
        System.out.println("Synchronized correct: "
                + (synchronizedSum == expected));
```

```
        System.out.println("Thread-local correct: " + (local ==
expected));
    }
}
```

Output:



Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What is a race condition, and why does it occur here?

   Ans.

   A race condition occurs when multiple threads access and modify shared data concurrently without proper synchronization. In our practical, multiple threads increment the same counter, so some updates may be lost due to overlapping read-modify-write operations.

2. What does the synchronized keyword do?

Ans.

The `synchronized` keyword protects shared data by allowing only one thread at a time to execute a synchronized block or method guarded by the same monitor. It also provides memory-visibility guarantees between threads using that monitor.

3. What are the states in a thread's life cycle?

Ans.

Java has six thread states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED. A thread progresses through these states depending on its execution, synchronization, waiting conditions, and completion.